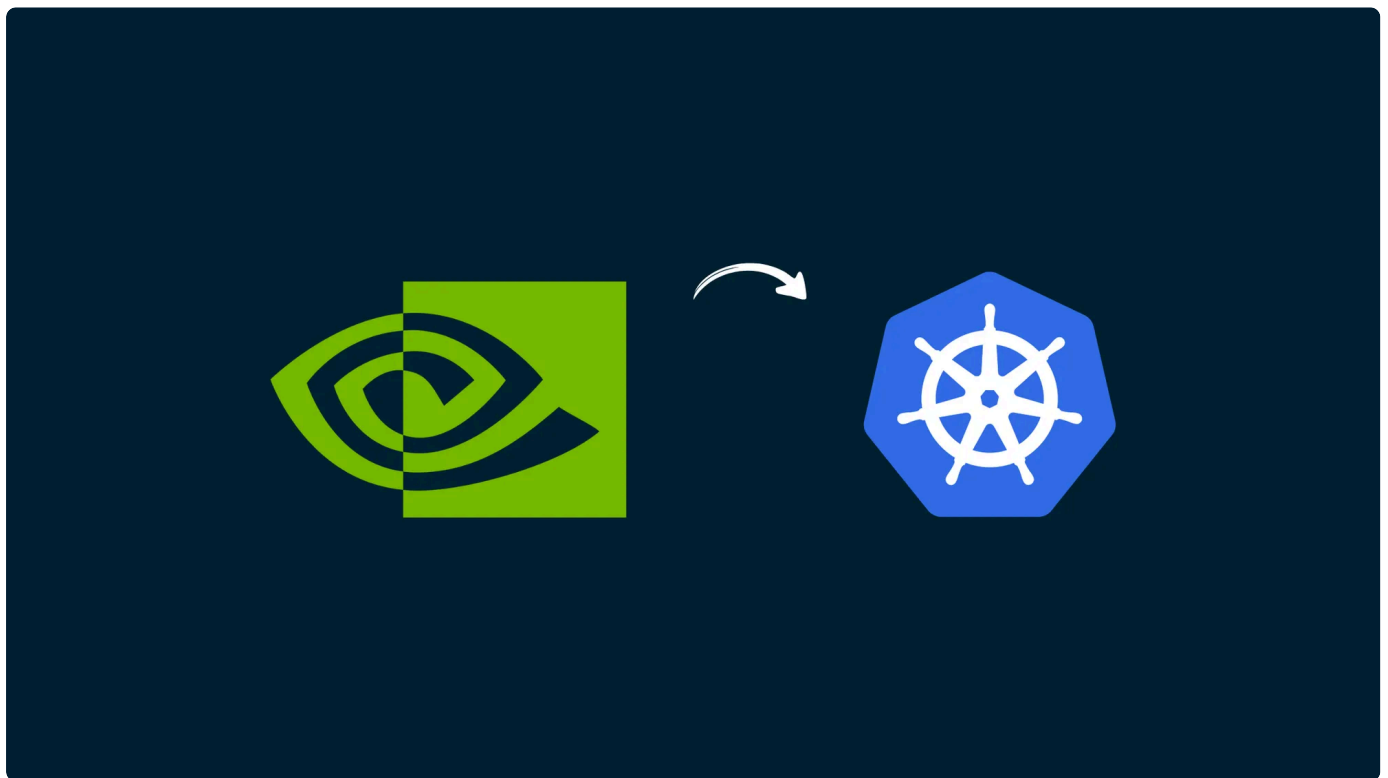


How to Setup GPU Operator on Kubernetes (Detailed Guide)



Bibin Wilson, Aswin Vijayan
Mar 4, 2026 8 min read



Looking for a guide to setup GPU operator on Kubernetes or to understand how to use GPUs with Kubernetes? This blog is for you.

By the end of this blog, you will have a clear understanding of:

- Need for GPU operator on Kubernetes
- Setting up NVIDIA GPU Operator on a Kubernetes cluster
- Verify if Kubernetes detects GPUs
- How to deploy a real GPU-based workload to validate the full stack.

DevOpsCube Newsletter

Subscribe to get tips and guides on DevOps, MLOps, Kubernetes, and more—delivered straight to your inbox!

Subscribe



Lets get started.

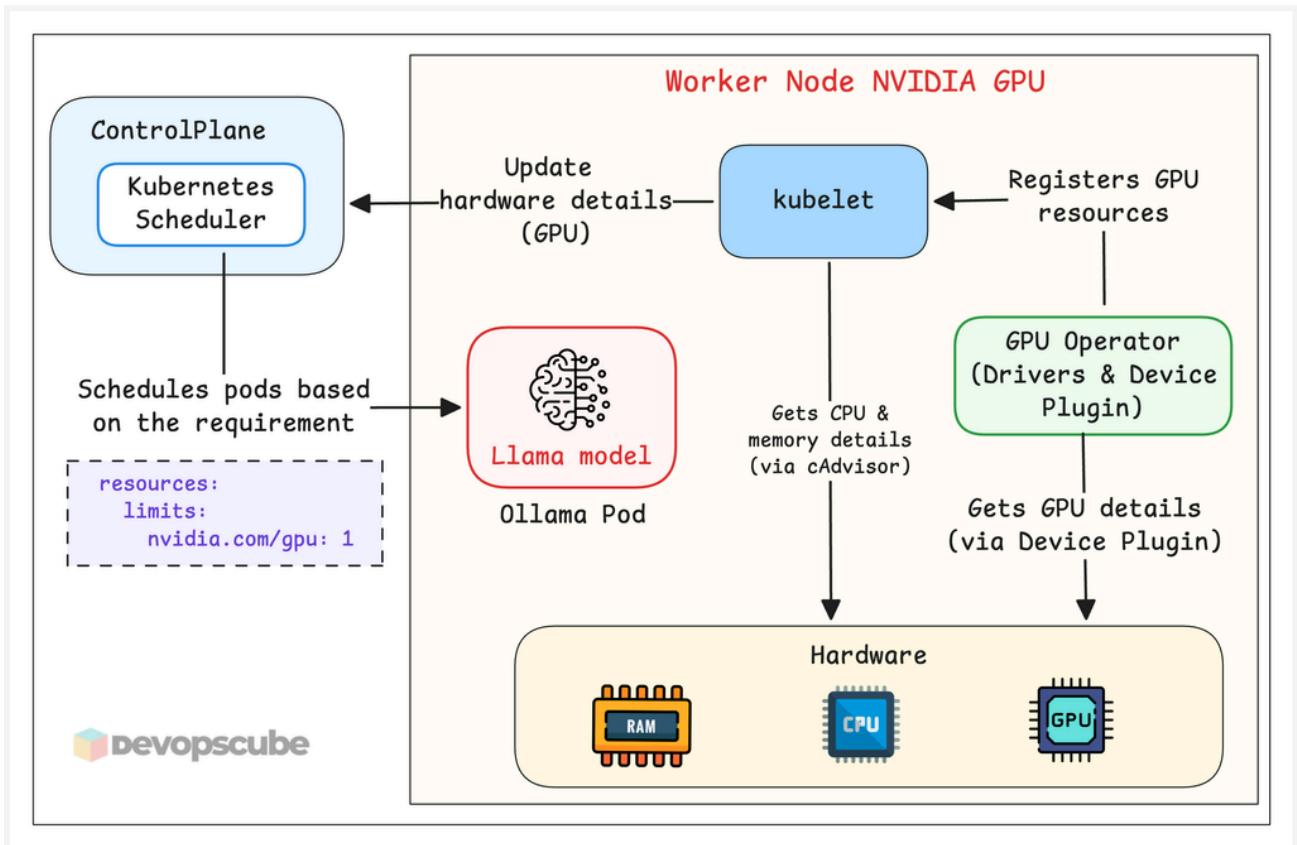
Why Kubernetes Can't See Your GPU

By default, Kubernetes will only have knowledge about CPUs and memory.

If you are provisioning GPU nodes for Kubernetes, it will not have knowledge about whether a GPU is attached to the node. The reason os GPU's are vendor specific hardware (NVIDIA, AMD, intel etc.

Kubernetes Device Plugins solves this by letting Kubernetes know about the GPU node details. Meaning, it helps hardware vendors (like NVIDIA or AMD GPUs) register their devices with the kubelet.

Now, installing device plugin and drivers for the GPU nodes manually is a complex task. This is where Kubernetes GPU operators come in. The GPU Operators will automate the installation of device plugins, drivers, Node Feature Discovery, runtimes, etc.



Here is an important thing.

There is no standard operator for GPU's. You have to choose the **operator based on the hardware** you are using. For example, if the GPU you are using is NVIDIA, you have to install the **NVIDIA GPU Operator** (one we are using in this guide).



Note: If you are using GPU nodes of managed Kubernetes services like EKS, AKS, DOKS, they may have pre installed GPU drivers on the GPU nodes.

In that case, the NVIDIA GPU Operator will skip the driver installation.

How GPU Operator works

Before we get in to hands-on lets understand how a GPU Operator works.

Here is how it works.

- Kubelet collects the hardware details of the node (via cAdvisor) and sends them to the Scheduler. Kubelet has no awareness about the GPU at this stage.
- Once we install the GPU operator, it installs a device plugin that collects the GPU details and registers `nvidia.com/gpu` as an extended resource with kubelet.
- Once the GPU details are registered, you can deploy GPU based applications and request for GPU resources (For example, llama model).

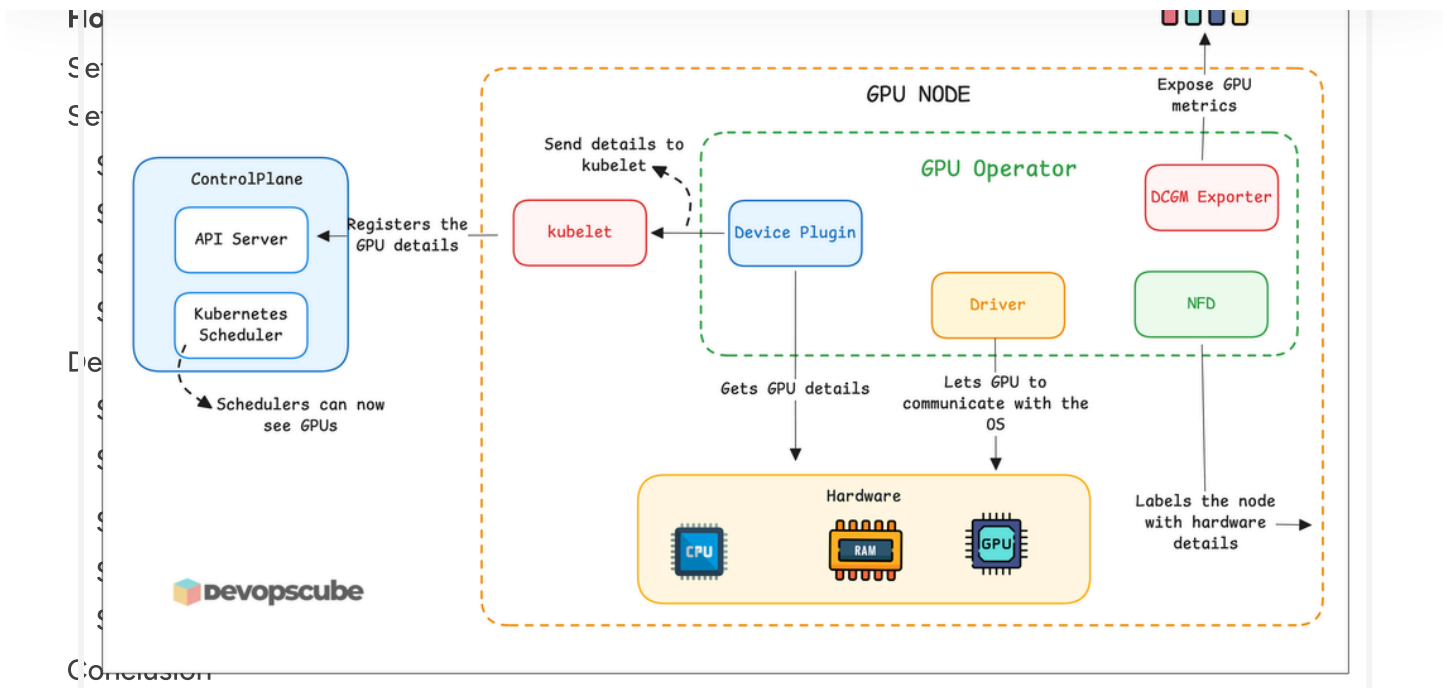


kubelet does NOT query GPU hardware directly. It relies entirely on the device plugin to discover and register GPU resources.

The following diagram illustrated how GPU nodes gets exposed to Kubernetes.



DevOps Deals



In the diagram, not all operator components are shown; only a few important components are shown.

These are the components that will be deployed by the NVIDIA GPU Operator.

- `gpu-operator` - Main controller of the operator which install and manages all components.
- `nvidia-driver-daemonset` - Install the NVIDIA GPU drivers on each GPU node so the node can use the GPU.
- `gpu-operator-node-feature-discovery.*` - Detects hardware details and adds as node labels.
- `nvidia-container-toolkit-daemonset` - Runs on all nodes and installed GPU runtimes for containers to use GPUs.
- `nvidia-cuda-validator` - Runs tests on the nodes to verify if drivers, plugins, and runtimes are installed on the node.
- `nvidia-dcgm-exporter` - This exports GPU metrics which can be scraped using Prometheus.
- `nvidia-device-plugin-daemonset` - This is the component that registers the GPU details to kubernetes
- `nvidia-operator-validator` - Checks the health of GPU operator

Let's start the Nvidia GPU Operator setup.

Setup Prerequisites

Below are the prerequisites required for this blog.

- Kubernetes cluster with GPU-enabled nodes
- kubectl
- Helm

Set Up the NVIDIA GPU Operator

Follow the steps given below to setup the operator.

Step 1: Label and Taint GPU Nodes

In mixed clusters (GPU + non-GPU nodes), you must prevent non-GPU workloads from consuming GPU node resources. We can use taints and labels to enforce this segregation.

Use the following command to add a taint and label the node.



Update your GPU nodes name in below command before running it.

```
kubectl taint node <your-gpu-node-name> nvidia.com/gpu=present:NoSchedule  
  
kubectl label node <your-gpu-node-name> node-type=gpu
```

This is one of the best practises for GPU nodes.



In managed GPU nodes (EKS, AKS etc the taints

`nvidia.com/gpu=present:NoSchedule` will be added as default.

Step 2: Add Nvidia GPU Operator Helm Repo

We are going to use Helm to install the Nvidia GPU operator, lets add the Helm repo for the chart.

Run the following command to add the repo and update the Helm repos.

```
helm repo add nvidia https://helm.ngc.nvidia.com/nvidia \  
  && helm repo update
```

And run the following command to verify if the NVIDIA repo has added.

```
helm search repo nvidia
```

You will see the following charts.

NAME	CHART VERSION	APP VERSION	DESCRIPTION
nvidia/nvidia-device-plugin	0.9.0	0.9.0	

nvidia/nvidia-dra-driver-gpu	25.12.0	25.12.0
nvidia/cybersecurity-dfp	0.2.1	23.07
nvidia/cybersecurity-sp	0.1.0	23.07
nvidia/deepstream-its	0.2.0	1.0
nvidia/dps	0.7.8	0.7.8
nvidia/dps-bmc-simulator	0.7.8	0.7.8
nvidia/ds-face-mask-detection	1.0.0	1.2
nvidia/ds-lipactivity	0.0.1	0.0.1
nvidia/fed-svr-3	0.9.0	1.0
nvidia/fed-wrk-3	0.9.0	1.0
nvidia/gpu-operator	v25.10.1	v25.10.1
nvidia/harbor-reef-operator	1.0.1	1.0.0
nvidia/isaac-lab-teleop	2.2.0	0.0.0
nvidia/k8s-nim-operator	3.0.2	3.0.2
nvidia/network-operator	25.10.0	v25.10.0
nvidia/nspect_test_policy_org_chart	1	1.16.0
nvidia/nvsm	1.0.1	1.0.1
nvidia/tensorrt-inference-server	1.0.0	1.0
nvidia/tensorrtinferenceserver	1.0.0	1.0
nvidia/tritoninferenceserver_aws	0.1.0	1.16.0
nvidia/video-analytics-demo	0.1.9	1.2
nvidia/video-analytics-demo-14t	0.1.3	0.1.3

Step 3: Install Nvidia GPU Operator

Let's install the Nvidia GPU Operator.

The GPU operator handles everything like installing and managing drivers, plugins, runtimes, etc.



The Nvidia GPU Operator Helm chart has toleration for the taint we added in the previous step by default. So there is no need for any additional configuration.

Now, use the following Helm command to install the operator.

```
helm install --wait gpu-operator \
  -n gpu-operator --create-namespace \
  nvidia/gpu-operator
```

Then, run the following command to check if the pods are up and running.

```
kubectl get po -n gpu-operator
```

You will get the following output.

```
gpu-feature-discovery-kbcth          1/1    Running    0          43s
gpu-operator-7569f8b499-dgzql        1/1    Running    0          57s
gpu-operator-node-feature-discovery-gc-55ffc49ccc-6bvmc 1/1    Running    0          57s
gpu-operator-node-feature-discovery-master-6b5787f695-1g584 1/1    Running    0          57s
gpu-operator-node-feature-discovery-worker-f871c      1/1    Running    0          57s
gpu-operator-node-feature-discovery-worker-nggs5      1/1    Running    0          57s
nvidia-driver-daemonset-dgzql        1/1    Running    0          57s
nvidia-container-toolkit-daemonset-92v1v             1/1    Running    0          44s
nvidia-cuda-validator-6pwtz          0/1    Completed  0          37s
nvidia-dcgm-exporter-192vx          1/1    Running    0          43s
nvidia-device-plugin-daemonset-5t52d 1/1    Running    0          44s
nvidia-operator-validator-4wcc6      1/1    Running    0          44s
```

Step 4: Verify GPU Detection

Now, let's verify if the GPU nodes are detected by Kubernetes.

Run the following command to check for the GPU capacity.



Update your GPU nodes name in below command before running it.

```
kubectl describe node <gpu-node-name> | grep -A6 "Capacity"
```

You will get the following output.

```
Capacity:
  cpu:                24
  ephemeral-storage:  742911020Ki
  hugepages-1Gi:      0
  hugepages-2Mi:      0
```

```
memory:          247413472Ki
nvidia.com/gpu:  1
```

You can see the `nvidia.com/gpu=1`, which means 1 GPU is available. The Nvidia GPU operator setup is completed now.



Debugging tip: If `nvidia.com/gpu` is missing, the device plugin is not running correctly. Check logs with: `kubectl logs -n gpu-operator -l app=nvidia-device-plugin-daemonset`

Next we will deploy a GPU based workload and schedule it on the GPU node.

Deploy a GPU Workload: Ollama + Llama

Let's validate the full stack by deploying a real GPU workload.

We will use **Ollama**, a lightweight runtime for open-source LLMs to run Llama 3.2 (3B parameters) on our GPU node.

Let's start the Ollama deployment.

Step 1: Add Ollama Helm Repo

To install Ollama, we will use a community Helm chart.

Run the following command to add the repo and update the Helm repos.

```
helm repo add otwld https://helm.otwld.com/
```

And run the following command to verify if the NVIDIA repo has added.

```
helm search repo ollama
```

You will get the following output.

NAME	CHART VERSION	APP VERSION
otwld/ollama	1.43.0	0.16.1

Step 2: Create Custom Helm Chart

To install Ollama, we need to make some configuration changes in the values file, like enabling GPU, adding toleration, selecting a node, etc.

Below is the custom values file we used to deploy Ollama.

Create a `ollama.yaml` file and copy the following content.

```
ollama:
  gpu:
    enabled: true
    type: nvidia
    number: 1

  models:
    pull:
      - llama3.2:3b

nodeSelector:
  node-type: gpu

tolerations:
  - key: "nvidia.com/gpu"
    operator: "Exists"
    effect: "NoSchedule"

persistentVolume:
  enabled: true
  size: 30Gi

resources:
  limits:
    nvidia.com/gpu: 1
```

If you want to check more configurations in the default values file, use the following command to get the values files.

```
helm show values otwld/ollama > values.yaml
```

Step 3: Deploy Ollama

Lets use the custom values file and install Ollama.

Use the following command to install it.

```
helm install ollama otwld/ollama \
  --namespace ollama \
  --create-namespace \
  -f ollama.yaml
```

Once the command is run, use the following command to verify if the pods are up and running.

```
kubectl get po -n ollama
```

You will get the following output.

NAME	READY	STATUS	RESTARTS	AGE
ollama-cc9b55478-v9ddq	1/1	Running	0	7m50s

Step 4: Verify GPU Usage

To verify if the pod is using a GPU, use the following command.

```
kubectl exec -n ollama deployment/ollama -- nvidia-smi
```

You can see the GPUs name as shown below.

```

→ ~ kubectl exec -n ollama deployment/ollama -- nvidia-smi
Tue Mar  3 12:58:26 2026

+-----+
| NVIDIA-SMI 575.57.08                  Driver Version: 575.57.08          CUDA Version: 12.9     |
+-----+-----+
| GPU  Name                Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |
|                                           | MIG M.         |
+-----+-----+
|  0   NVIDIA H100 80GB HBM3           On   | 00000000:00:09.0 Off |           0         |
| N/A   34C    P0              69W / 700W |  0MiB / 81559MiB |      0%      Default |
|                                           | Disabled       |
+-----+-----+

Processes:
+-----+
| GPU  GI   CI           PID   Type   Process name                      GPU Memory |
| ID   ID   ID                                   Usage     |
+-----+
| No running processes found |
+-----+

```

Step 5: Query the Model

Now, Ollama pods are up and running, let's send a query to the llama model running on Ollama.

Since the services are in Cluster IP, we will expose them using port forwarding.

Use the following command to port forward the service.

```
kubectl port-forward svc/ollama 11434:11434 -n ollama
```

Once it's port forwarded, run the following command with the query as shown below.

```

curl -s http://localhost:11434/api/generate \
-d '{
  "model": "llama3.2:3b",
  "prompt": "What do you know about Kubernetes?",
  "stream": false
}' | jq -r '.response'

```

You will get an output similar to what is shown below.

```
➔ ~ curl -s http://localhost:11434/api/generate \
-d '{
  "model": "llama3.2:3b",
  "prompt": "What do you know about Kubernetes?",
  "stream": false
}' | jq -r '.response'
```

Kubernetes (also known as K8s) is an open-source container orchestration system for automating the deployment, and is now maintained by the Cloud Native Computing Foundation (CNCF).

Here are some key features and concepts of Kubernetes:

Key Features:

- Container Orchestration:** Kubernetes manages the lifecycle of containers, including creation, scaling, and termination.
- Declarative Configuration:** Users define the desired state of their application using YAML or JSON.
- Self-healing:** If a container fails or is terminated, Kubernetes automatically recreates it to ensure the desired state is maintained.
- Resource Management:** Kubernetes provides features for managing resources such as CPU and memory, ensuring efficient utilization.
- Scalability:** Kubernetes can scale applications horizontally (by adding more replicas) or vertically (by adjusting resource requests).
- High Availability:** Kubernetes ensures that applications are available even in the event of component failures.

Key Concepts:

- Pods:** A group of one or more containers that share the same network namespace and resources.
- Replica Sets:** Ensure a specified number of replicas (i.e., copies) of a pod are running at any given time.
- Deployments:** Manage the rollout of new versions of an application by creating a new deployment and updating the existing pods.

Conclusion

You now have a complete picture of GPU scheduling on Kubernetes.

You have learned about setting up a GPU operator on Kubernetes and deployed a real LLM on a GPU node.

When it comes to MLOps, understanding GPU node on Kubernetes is very important as many ML workloads need GPU based clusters.

Having said that, getting GPUs working is step one. There are different strategies in managing GPUs in Kubernetes for the efficient use of GPU resources. For example, **GPU Time-Slicing**, **Multi instance GPU (MIG)** etc..

We will look at in the upcoming hands-on blogs.

If you are facing issues when setting up GPU operator, do let us know in the comments.



ABOUT THE AUTHOR

Bibin Wilson

Bibin Wilson (authored over 300 tech tutorials) is a cloud and DevOps consultant with over 12+ years of IT experience. He has extensive hands-on experience with public cloud platforms and Kubernetes.

Featured



Python For DevOps: Guide for DevOps Engineers

19 Jan 2026



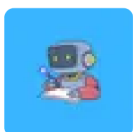
Kubernetes Tutorial For Beginners: 75 Comprehensive Guides

08 Jan 2026



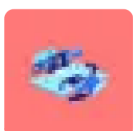
Become A DevOps Engineer in 2026: [A Practical Roadmap]

06 Jan 2026



Kubernetes Built-in AI/ML Features You Should Know

05 Jun 2025



Jenkins Tutorial For Beginners: 21+ Practical Guides

01 Apr 2023

Latest



Helm Chart Tutorial: A Simple Guide for Beginners

19 Jun 2026



SLSA Provenance Creation using GitHub Actions

16 Jun 2026



How to Back Up and Restore a WordPress Site on Bluehost

15 Jun 2026



How to Create AWS EKS Cluster Using eksctl

04 Jun 2026

Kommentar schreiben

Senden

GIF

Bitte gib deine E-Mail Adresse ein.

Ich habe ein Konto

E-Mail zur Überprüfung.

Nutzername

Wir senden keine Marketing- oder Werbe-E-Mails.

Senden

FastComments.com

READ NEXT

Helm Chart Tutorial: A Simple Guide for Beginners



SLSA Provenance Creation using GitHub Actions



How to Back Up and Restore a WordPress Site on Bluehost



How to Create AWS EKS Cluster Using eksctl



Set Up WordPress on Bluehost in 9 Minutes (Detailed Beginners Guide)



DevOpsCube – Award-winning Blog on DevOps, SRE, MLOps, Cloud & CI/CD

Navigate

Tutorials

Kubernetes Tutorial

ArgoCD Tutorial

Jenkins Tutorial

Roadmaps

DevOps Engineer Roadmap

DevOps To MLOps

Kubernetes Learning RoadMap

Python For DevOps

Golang For DevOps

Git For DevOps

Shell Scripting For DevOps

Certification Guides

CKA Exam Guide

CKAD Exam Guide

CKS Exam Guide

KCNA Exam Guide

KCSA Exam Guide

Prometheus Exam Guide

Argo Certification Guide

Istio Certification Guide

Newsletter

Our Courses

Container Fundamentals

Kubernetes & CKA Preparation Course

Social

 Youtube

 Twitter

